



CL-2001 Data Structures Lab # 9

Objectives:

- Recursion
- BST Linked List

Note: Carefully read the following instructions (*Each instruction contains a weightage*)

1. There must be a block of comments at start of every question's code by students; the block should contain brief description about functionality of code.
2. Comment on every function about its functionality.
3. Use understandable name of variables.
4. Proper indentation of code is essential.
5. Write a C++ statement(s) for each of the following task one after the other, in the same order.
6. Make a Microsoft Word file and paste all of your C++ code with all possible screenshots of **every task output in MS word and submit .cpp file with word file.**
7. Make separate .cpp files for all tasks and use this format **22F-1234_Task1.cpp.**
8. First think about statement problems and then write/draw your logic on copy.
9. After copy pencil work, code the problem statement on MS Studio C++ compiler.
10. At the end when you done your tasks, attached C++ created files in MS word file and make your submission on Google classroom. (Make sure your submission is completed).
11. Please submit your word file in this format **22F-1234_L1.docx**
12. Do not submit your assignment **after the deadline.**
- 13. Do not copy code from any source otherwise you will be penalized with negative marks.**

Problem: 1 | Linked list-based Binary Search Tree

Implement Binary Search tree with the following function mentioned below in drive code.

Driver Code:

```
#include<iostream>
using namespace std;

struct node
{
    int data;
    node* left;
    node* right;
}*root

class BST
{
public:
    BST()
    {
        root = nullptr;
    }
    void inorder(node* root){}
    void preorder(node* root){}
    void postorder(node* root){}
    void insert(node* root, int data){}
    void find(node* root, int key){}
    void del(node* root, int key){}
};

void main()
{
    int choice, num;
    BST bst;
    while (1)
    {
        cout << "-----" << endl;
        cout << "Operations on BST" << endl;
        cout << "-----" << endl;
        cout << "1.Insert Element " << endl;
        cout << "2.Delete Element " << endl;
        cout << "3.Inorder Traversal" << endl;
        cout << "4.Preorder Traversal" << endl;
        cout << "5.Postorder Traversal" << endl;
        cout << "6.Quit" << endl;
        cout << "Enter your choice : ";
        cin >> choice;
        switch (choice)
        {
            case 1:
                cout << "Enter the number to be inserted : ";
                cin >> num;
                bst.insert(root, num);
            case 2:
                cout << "Enter the number to be deleted : ";
                cin >> num;
```

```
        bst.del(num);
        break;
    case 3:
        cout << "Inorder Traversal of BST:" << endl;
        bst.inorder(root);
        cout << endl;
        break;
    case 4:
        cout << "Preorder Traversal of BST:" << endl;
        bst.preorder(root);
        cout << endl;
        break;
    case 5:
        cout << "Postorder Traversal of BST:" << endl;
        bst.postorder(root);
        cout << endl;
        break;
    case 6:
        exit(1);
    default:
        cout << "Wrong choice" << endl;
    }
}
system("pause");
}
```

Problem: 2 | Linked list-based Binary Search Tree | Cricket Player Ranking System

You are a software developer for International Cricket Council and company wants to improve its ranking system by creating a leaderboard that tracks player performance. This leaderboard will be represented as a list, where each node represents a player.

The node associated with each player contains the following information:

1. **playerId**: A unique ID associated with the player.
2. **playerName**: The player's username.
3. **score**: The player's score, which is an integer.

Your task is to implement the following methods:

1. **addPlayer(playerId, playerName, score)**: This function adds a new player to the leaderboard. The new player should be inserted in the correct position according to their score. If two players have the same score, the player added most recently should be ranked higher.
2. **removePlayer(playerId)**: This function removes a player from the leaderboard. If the player is not found, it should return a message indicating this.
3. **updateScore(playerId, score)**: This function updates the score of a player and repositions them in the leaderboard according to their new score.
4. **displayPlayers(n)**: This function print a list of the n number of players (player IDs and scores). it should contain n number of players.



Problem: 3 | Linked list-based Binary Search Tree | Cricket Player Ranking System

Write a function to count the leaf nodes, findMax and findMin in a BST.

Problem: 4 | Linked list-based Binary Search Tree | Home Work

Implement a flip BST in such a way that the left sub tree nodes will be shifted to right side of the tree and same case for right sub tree.